

Assignment 1

Section - A

Solⁿ 1) Abstraction is the process of hiding hiding complex implementation details & showing only essential features of a system.

We use it because:

- Reduces system complexity
- Makes design & development easier
- Enables modularity.

Solⁿ 2) Levels of Abstraction:

- (i) Hardware Level - Physical components (CPU, memory)
- (ii) Machine level - Binary instructions
- (iii) Assembly level - Symbolic instructions
- (iv) High-level languages - C, C++, Java
- (v) Application level - End user programs

Example:

Print "Hello" → Python code → Compiled to assembly → converted to machine code → executed by hardware

Solⁿ 3) Functional Units:

- (i) Input Unit: take input from user
- (ii) Memory Unit: store data & instructions

- (iii) Arithmetic Logical Unit (ALU): Performs calculations
 (iv) Control Unit (CU): controls execution of instructions
 (v) output Unit: Display output

10th 4) ALU v/s CU:

ALU:	CU:
- Performs arithmetic & logical operations Fetches Performs calculations & comparisons	- controls & coordinates all operations - Fetches, decodes & executes instructions

Register v/s Cache:

Register	Cache
- Smallest, fastest storage inside the CPU - Processor utilizes registers to store operands that it is currently processing	- Slightly larger, stores frequently used data - Processor utilizes cache to store recently utilized data

Q5) Von-Neumann Model:

- A computer architecture where CPU, memory & I/O (input/output) are connected & instructions / data share the same memory.

Stored-Program concept:

↳ instructions are stored in memory like data & executed sequentially

Q6) Limitation caused due to a single path (bus) b/w CPU & memory

Key points:

- Data & instructions share same bus
- Causes delay in execution
- Limits system performance

Q7) Number of transistors on a chip doubles approximately every 18-24 months

↳ Impact:

- increased performance
- Reduced cost
- Smaller & powerful devices (more powerful)

Q10) Instructions Per Cycle (IPC):

- IPC is the number of instructions ~~per cycle~~ in a CPU executes in one cycle
- Higher IPC $\rightarrow$ Better CPU performance
- calculated by dividing the ~~total number~~ number of instructions executed by the number of clock cycles required to ~~execute~~ execute them:

$$IPC = \frac{\text{Number of instruction}}{\text{Number of clock cycles}}$$

SOP 9) Execution Time: Total time to run a program

clock Rate: Number of cycles per second

CPI (cycles per Instruction): avg cycle per ~~instructions~~ instruction

IPC: Instructions per cycle

Relation: Execution Time = $\frac{\text{instructions} \times \text{CPI}}{\text{clock Rate}}$
 $IPC = 1 / CPI$

(P10)

Solⁿ 10) Definitions:

Bit: Smallest Unit (0 or 1)

Byte: 8 bits

Signed: Represents +ve or -ve value

Unsigned: only +ve values

Example:

8 bit unsigned: 0 to 255

8 bit signed: -128 to 127

Solⁿ 11) Basic operations are:

- Arithmetic (addition, subtraction, multiplication)
- Logical (AND, OR, NOT)
- Comparison ($=$, $>$, $<$)
- Data movement (Load / store)

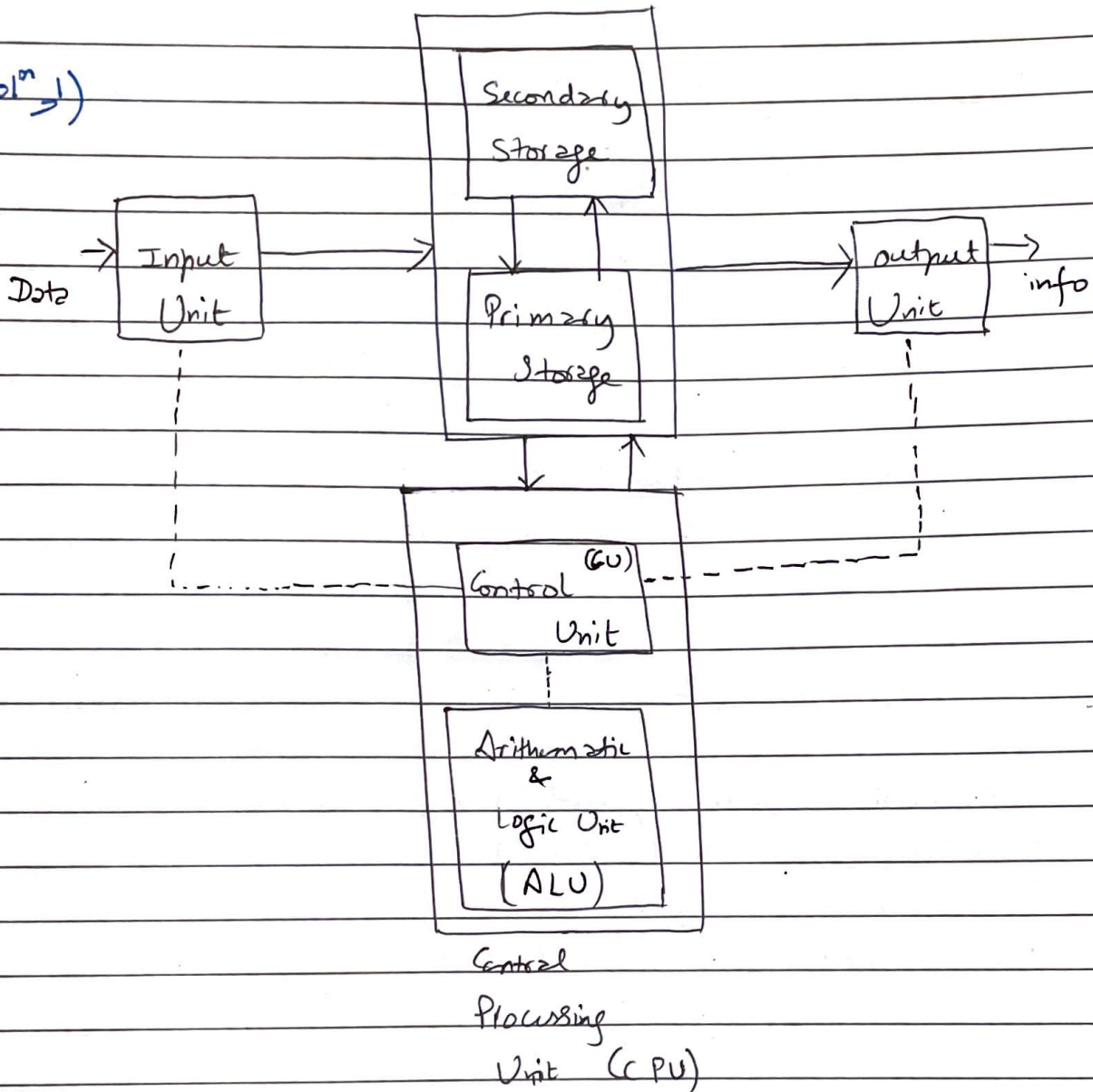
Solⁿ 12) Character Representation:

- ASCII (7/8-bit)
- Unicode (support multiple languages)

Decimal Representation:

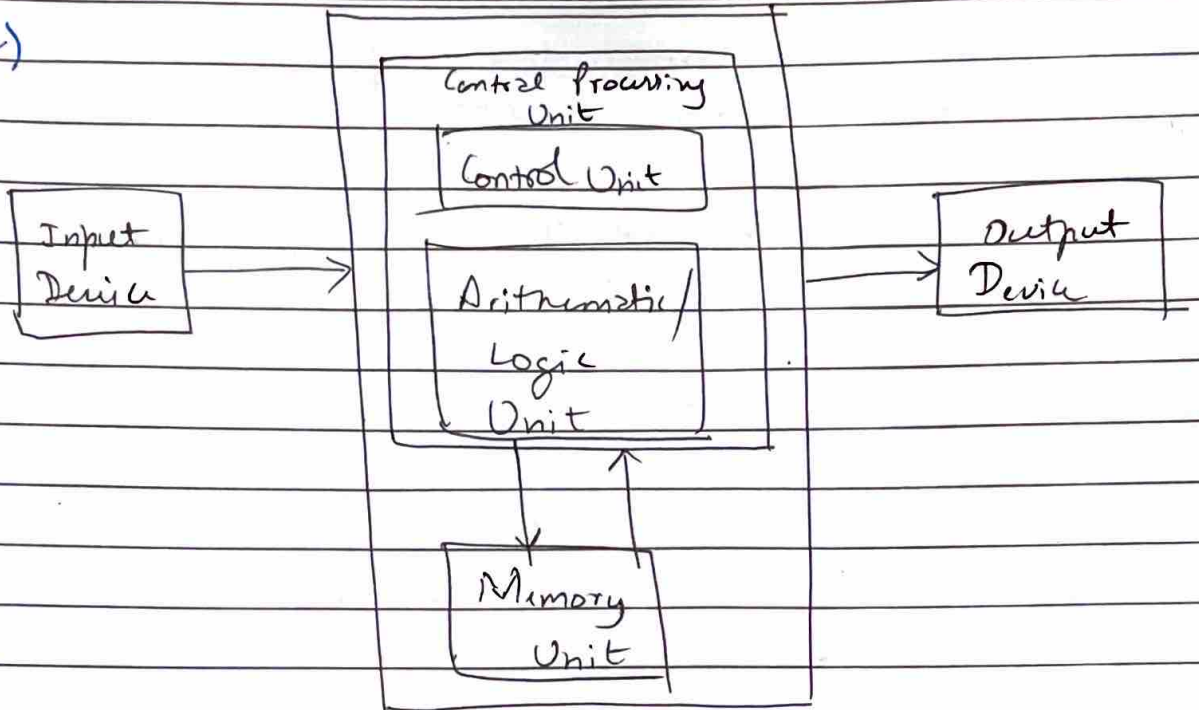
- Fixed-point (simple representation)
- Floating-point (scientific, eg. IEEE754)

Section B

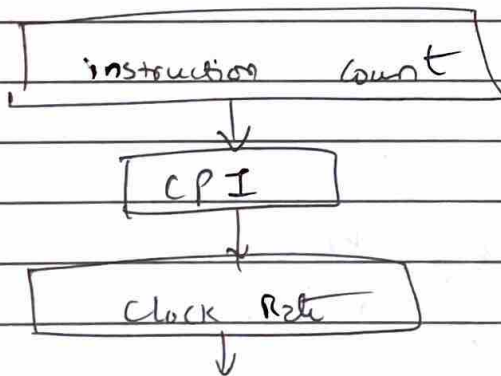
Solⁿ → 1)Explanation:

- Input unit takes data, CPU processes it using ALU & CU
- Memory stores data & instructions
- Output unit displays results, buses connect all unit

(P70)
→

Solⁿ 2)

- Explanation:
- single memory stores both data & instructions
 - CPU communicates with memory & input/output via 2 common bus
 - Basis of most modern computers

Solⁿ 3)

CPU Execution Time

$$T_{PC} = \frac{1}{CPI}$$

Explanation: CPU time depends on instruction count, CPI & clock rate.

Date.....

- Higher clock rate $\downarrow$ time, high CPI $\uparrow$ time
- IPC is inverse of CPI

$\rightarrow$ IPC $\uparrow$ = better performance

Solⁿ 4) Diagram:

Decimal:	-8	-7	-6	-5	-4	-3	-2	-1	0
Binary:	1000	1001	1010	1011	1100	1101	1110	1111	0000

1	2	3	4	5	6	7
0000	0001	0010	0011	0100	0101	0110

Explanation:

- 4-bit 2's complement ~~range~~ range -8 to +7
- MSB (leftmost bit) indicates sign
- Used to represent -ve numbers efficiently.

Section C

Solⁿ 1) Given, CPU A: 3 GHz, CPI 1.5
CPU B: 2.2 GHz, CPI 1.0

Formula:

$$\text{CPU Time} = \frac{(\text{instruction count} \times \text{CPI})}{\text{clock rate}}$$

$$\text{CPU A} = \frac{1.5}{3.0} = 0.5$$

$$\text{CPU B time} = \frac{1.0}{2.2} = 0.45$$

Spiral

Date.....

$\therefore$ CPU B has lower execution time, hence faster

$\hookrightarrow$ Reason: lower CPI outweighs lower clock rate

Solⁿ 2) Effect of cache miss:

- cache miss $\rightarrow$ CPU waits (stall cycles $\uparrow$)
- IPC decreases (fewer instructions per cycle)
- execution time $\uparrow$

Fixes:

- $\uparrow$ cache size
- Improve cache locality
- Use multilevel cache

Solⁿ 3) Given,

Range: 200 values

Precision: 0.1 $\rightarrow$ need decimal accuracy

Step 1: choice

- use fixed point representation

Step 2: Justification

- fixed point is sufficient for limited range
- more efficient & faster than floating point
- less memory usage & simple hardware

solⁿ 4) Critique of given statement are mentioned below

→ Moore's law refers to transistor transistor count, not ~~direct~~ direct performance

→ Performance gains ~~are~~ now limited due to:

- ↳ Power & heat constraints
- ↳ slower clock speed scaling

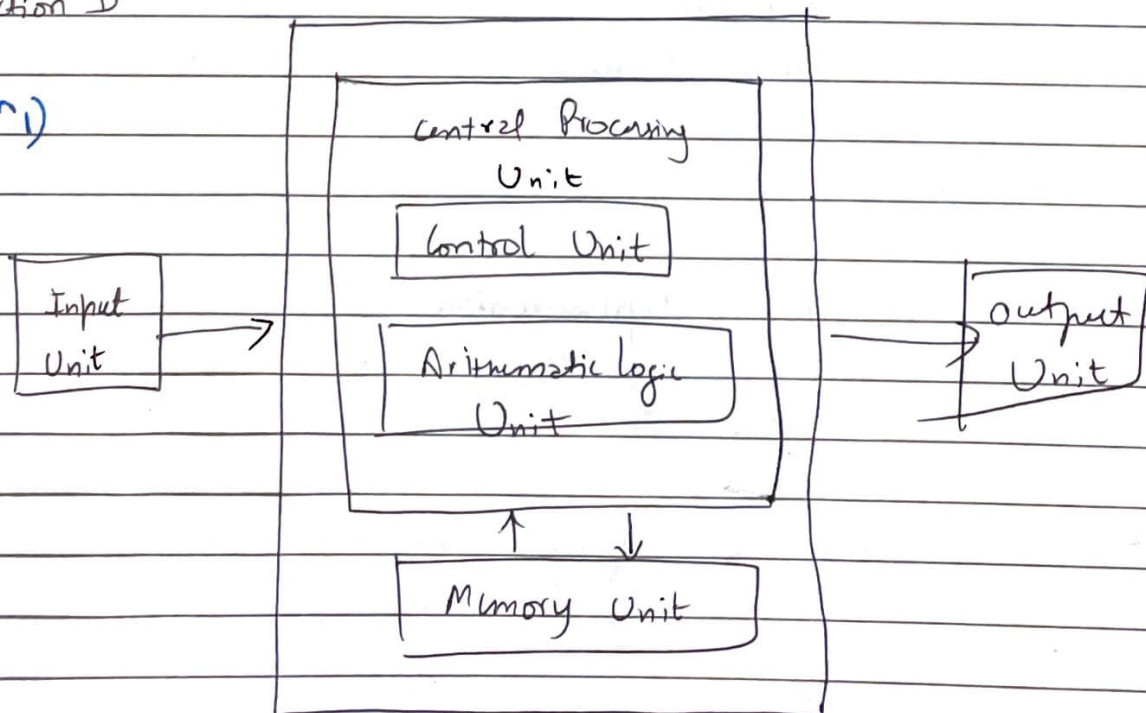
→ Modern improvements depend on:

- ↳ Parallelism (multi-core CPUs)
- ↳ Better architecture

Conclusion, Moore law alone does not guarantee automatic performance doubling today

Section D

solⁿ 1)



Von-Neumann Architecture

Design Decision:

↳ Single memory for data & instructions

Explanation:

- Simplifies system design & reduces hardware cost
- Not leads to von-Neumann bottleneck due to shared bus

solⁿ 2) Performance Relation:

$$\text{CPU Time} = (\text{Instruction Count} \times \text{CPI}) / \text{clock rate}$$

Explanation:

- Performance improves when CPU time decreases
- Depends on IC, CPI & clock rate

Improvement:

- Reduce CPI using pipeline & better cache
- Reduce IC using optimized algorithms
- Increase clock rate using better hardware

solⁿ 3) Represent Number:

Let's take +5 & +4 (4-bit system)

⇒ +5 ⇒ 0101, +4 ⇒ 0100

Addition:

$$\begin{array}{r} 0101 \\ + 0100 \\ \hline 1001 \end{array}$$

Interpretation:

1001 in 2's complement = -7

Overflow Check:

~ Adding two positive numbers $\rightarrow$ result should be true
~ But result is -ve $\rightarrow$ overflow occurs

Conclusion:

Overflow happens when result exceeds representable range.